

Backend Engineering Standards — Climate & Disaster Platform

Sep 28, 2026 · @Giddy

You are the backend engineering agent for the Climate Intelligence & Disaster Management Platform (Rivers State, Nigeria). Everything below is not reference material — it is your operating instructions for every piece of backend code you write, from the first migration to the last endpoint. Follow every rule in this document exactly; where a rule conflicts with something faster, more familiar, or more common practice elsewhere, the rule in this document wins.

This is the adapted version of a backend engineering standard handed over from another project, reworked to fit this platform: geospatial (PostGIS) support, role + region-based access control in place of multi-tenant ownership, and the project's data-provenance discipline enforced at the validation layer.

You are building this for people who will read the code you write — some newer to this exact stack (Node.js/TypeScript/Express/TypeORM/PostGIS) even where experienced elsewhere. **Section 18 is not optional:** every feature you implement ships with documentation a beginner can follow — what you built, why, what to read to understand it, and a plain-language explanation. Skipping that is not "finishing later" — it is incomplete work.

1. Purpose, scope & the stack decision this locks in

This document adapts a backend engineering standard from another product (a food-delivery app's codebase, apparently built on top of an earlier school-management SaaS's actual patterns) so it fits the Climate Intelligence & Disaster Management Platform. It is for the Software Engineers building the API, and for any AI coding agent working in this codebase.

Handing over a working Node.js/TypeScript/Express/TypeORM standard settles the open question from the Approach & Architecture doc (section 17): **backend stack = Node.js + TypeScript + Express + TypeORM + PostgreSQL/PostGIS**, not ASP.NET Core. This is a reuse decision, not a fresh evaluation — the team already has a working, battle-tested standard in this stack, including one real production incident and its fix (section 11). Flagging this explicitly because the project brief left the choice open pending comparison; that comparison is now moot given this handover.

What carries over from the source guide largely unchanged: the layered architecture shape, the response envelope, error handling, and the router-mounting lesson — all domain-agnostic. What changes: the multi-tenant `ownerId` isolation pattern is replaced with a role + region-scoping access model (this platform has no tenants — it has one dataset, several roles, and region-restricted staff accounts); a new geospatial/PostGIS section is added (the source guide has none); validation is extended to enforce this project's `data_type` provenance tagging as a hard rule; file uploads are marked Future/deferred; and the email standard is fully rebranded and trimmed to two tones instead of three.

Out of scope for this document: database schema (see the Architecture & Backlog doc's ERD), infrastructure/deployment (Cloud Engineer's remit, section 18 of that doc), and frontend conventions.

2. Core philosophy

- **Strict TypeScript only.** No `any` without a written justification comment; `strict: true` in `tsconfig.json`. Run everything with `tsx`, never plain `node` — `node` silently ignores type errors that `tsx / ts-node` would catch at run time.
- **`npm run typecheck` is mandatory before any commit.** No file is considered done while `tsc --noEmit` reports errors, including in files the change didn't touch but broke.
- **SOLID, applied practically:** one responsibility per class/module; services depend on repository interfaces, not concrete DB calls; new disaster types or data sources extend the system without editing existing switch statements (open/closed); small, focused interfaces over one large `IDataService`.
- **Thin controllers, fat services.** A controller parses the request, calls one service method, and shapes the response. All business logic — including the never-fabricate-data checks — lives in services, so it can be unit tested without an HTTP layer.
- **Repository-only database access.** No controller or service ever imports `AppDataSource` or writes raw SQL/TypeORM query builder code directly — that only happens inside a repository (section 8).
- **Fail fast with typed domain errors.** Throw `AppError` subclasses (section 5), never a bare `Error` or a silently-swallowed `try/catch`.
- **No silent data invention, anywhere in the stack.** This is the platform's core commitment (project brief, section 8) and it is a coding rule, not just a data-team rule: no service, mock, seed script, or fallback value may present an estimated, interpolated,

or placeholder number as if it were observed. See section 13 for how this is enforced.

3. Layered architecture & folder structure

Every request flows through the same seven layers, in this order, every time:

```

Validation Middleware (Zod)
    ↓
Auth Middleware (Firebase ID token)
    ↓
Authorization Guard (role + region check)
    ↓
Controller (thin – parse request, call service, shape response)
    ↓
Service (business logic, singleton via static Instance)
    ↓
Repository (all DB + PostGIS access, extends BaseRepository<T>)
    ↓
Model / Entity (TypeORM)
    ↓
PostgreSQL / PostGIS

```

No layer is skipped and no layer reaches past its neighbor (a controller never calls a repository directly; a service never touches `req / res`).

Folder structure, organized by feature module rather than by layer, mirroring the platform's real domain instead of the source guide's food-ordering / school modules:

```

src/
├── modules/
│   ├── auth/           # login, register, refresh, password reset
│   └── users/          # user profile, role assignment, region
scope
├── admin-units/        # countries, admin_units (LGAs, states,
etc.)
├── disaster-events/    # disaster_events CRUD + geospatial queries
├── disaster-types/     # controlled taxonomy (section 11 of the
brief)
├── rainfall-observations/ # climate_observations /
weather_observations
│   ├── risk-assessments/ # risk scoring, flood_susceptibility_zones
│   └── alerts/          # alert issuance and delivery status

```

```

|   |─ data-sources/           # data_sources, datasets, import runs
|   |─ reports/               # user-submitted disaster reports (if in MVP
scope)
|       |─ *.controller.ts
|       |─ *.service.ts
|       |─ *.repository.ts
|       |─ *.entity.ts
|       |─ *.validation.ts    # Zod schemas
|       |─ *.routes.ts
|─ middleware/                # authMiddleware, authorise(), errorHandler
|─ common/
|   |─ errors/                # AppError + subclasses
|   |─ response/              # ApiResponse envelope helpers
|   |─ pagination/            # offset + cursor helpers
|   |─ geo/                   # geometry validation, SRID helpers (section
8)
|─ config/                    # env, data source (TypeORM) config
|─ app.ts                     # router mounting – see section 11's incident

```

Each module is self-contained end to end (its own controller through entity); shared cross-cutting code lives in `common/`, never duplicated per module.

4. Validation layer

All request input is validated with **Zod** at the edge, before it reaches a controller. A schema per endpoint, colocated in the module as `*.validation.ts`, wired in as middleware:

```
export const createDisasterEventSchema = z.object({
  body: z.object({
    disasterTypeId: z.string().uuid(),
    eventDate: z.coerce.date(),
    adminUnitId: z.string().uuid(),
    severity: z.enum(["low", "moderate", "severe",
"catastrophic"]).optional(),
    location: geoPointSchema, // { type: "Point", coordinates: [lon, lat] }
    dataType: z.enum(["OBSERVED", "DERIVED", "MODELLED", "ESTIMATED",
"SYNTHETIC"]),
    sourceId: z.string().uuid(),
    description: z.string().max(2000).optional(),
  }),
});
```

Project-specific hard rule — this is not optional and not just a data-team convention:

any write to `disaster_events`, `rainfall_observations` / `climate_observations`, `population_estimates` (if added), or `flood_susceptibility_zones` MUST include a valid `dataType` (`OBSERVED` | `DERIVED` | `MODELLED` | `ESTIMATED` | `SYNTHETIC`) and a `sourceId` that resolves to a real row in `data_sources`. A request missing either is rejected at validation with `400` before it ever reaches a service — the API makes it structurally impossible to write an untagged, unsourced number into these tables. `SYNTHETIC` data is additionally rejected outside a `NODE_ENV=development` or a dedicated demo-data endpoint (section 13).

Geometry input validation: geometry fields are accepted as GeoJSON (`Point` / `Polygon` / `MultiPolygon` / `LineString`) and validated for: correct GeoJSON shape, coordinate order [longitude, latitude] (GeoJSON's convention — easy to invert by mistake), coordinate ranges (`-180..180` , `-90..90`), and polygon ring closure. All geometry is assumed and enforced as **EPSG:4326 (WGS84)** — no other SRID is accepted at the API boundary; conversions happen at ingestion time if a source uses a different CRS (see the Architecture doc's data pipeline).

Import-pipeline format support: the rules above govern direct API writes (GeoJSON only, since those endpoints expect a JSON body). The generic import pipeline (Architecture doc, section 1) accepts a wider set of source formats — CSV, GeoJSON, Shapefile, and XLSX — and normalizes each to the same internal geometry representation during the pipeline's Geospatial Processing stage, before the row reaches promotion: GeoJSON input goes through `ST_GeomFromGeoJSON`, WKT input (common in spreadsheet-delivered boundary data, e.g. a `geometry_wkt` column) goes through `ST_GeomFromText(wkt,`

4326) . Parse XLSX workbooks with **exceljs** (actively maintained, without the prototype-pollution CVE history the plain `xlsx` /SheetJS npm package has had) — read one sheet per table, matching sheet names to the tables being imported (e.g. a `rainfall` sheet and a `lga_boundaries` sheet in one file, as in the delivered `rivers_state_rainfall_and_boundaries.xlsx` test fixture).

5. Response envelope & error handling

Carried over from the source guide largely unchanged — this layer is domain-agnostic. Every endpoint returns the same envelope shape:

```
interface ApiResponse<T> {
  success: boolean;
  data: T | null;
  message: string;
  errors?: { field?: string; message: string }[];
  meta?: { page?: number; pageSize?: number; total?: number; nextCursor?:
string };
}
```

Domain errors are thrown, never returned as values:

```
export class AppError extends Error {
  constructor(
    public readonly statusCode: number,
    message: string,
    public readonly errors?: { field?: string; message: string }[],
  ) { super(message); }
}

export class NotFoundError extends AppError {
  constructor(resource: string) { super(404, `${resource} not found`); }
}

export class ValidationError extends AppError {
  constructor(errors: { field?: string; message: string }[]) { super(400,
"Validation failed", errors); }
}

export class ForbiddenError extends AppError {
  constructor(message = "Not authorized for this region or role") {
super(403, message); }
}
```

```
export class UnprovenDataError extends AppError {
  constructor() { super(422, "Write rejected: missing or invalid
dataType/sourceId"); }
}
```

A single global error-handling middleware, mounted last in `app.ts`, catches every thrown error: `AppError` subclasses map to their own `statusCode` and shape the `errors` array; anything unrecognized is logged with full context and returned as a generic `500` (`message: "Something went wrong"`) — stack traces and internals never reach the client, even in development responses, only in server-side logs.

6. Controller & service layer standards

Controller — thin, no business logic:

```
export class DisasterEventController {
  static async create(req: Request, res: Response, next: NextFunction) {
    try {
      const result = await DisasterEventService.Instance.create(req.body,
req.user!);
      return res.status(201).json({ success: true, data: result, message:
"Disaster event recorded" });
    } catch (err) { next(err); }
  }

  static async list(req: Request, res: Response, next: NextFunction) {
    try {
      const { items, meta } = await
DisasterEventService.Instance.list(req.query, req.user!);
      return res.json({ success: true, data: items, message: "OK", meta });
    } catch (err) { next(err); }
  }
}
```

Service — singleton via `static Instance`, holds the business rules:

```

export class DisasterEventService {
  private static _instance: DisasterEventService;
  static get Instance() { return this._instance ??= new
DisasterEventService(DisasterEventRepository.Instance,
DataSourceRepository.Instance); }
  private constructor(private repo: DisasterEventRepository, private
sourceRepo: DataSourceRepository) {}

  async create(input: CreateDisasterEventInput, user: AuthUser) {
    const source = await this.sourceRepo.findById(input.sourceId);
    if (!source) throw new UnprovenDataError();           // sourceId must
resolve to a real source
    assertRegionAccess(user, input.adminUnitId);          // section 7
    return this.repo.create({ ...input, reportedByUserId: user.id });
  }

  async list(query: DisasterEventListQuery, user: AuthUser) {
    const scoped = applyRegionScope(query, user);         // section 7
    return this.repo.findPage(scoped);                     // section 8/10
- geometry omitted from list results
  }
}

```

Controllers never call a repository directly, and services never read `req / res` — a service takes plain inputs and an `AuthUser`, so it can be unit-tested with no HTTP mocking at all.

7. Access control model

This is the biggest structural change from the source guide. The source guide isolates data with a flat `ownerId` check per entity (`WHERE ownerId = req.user.id`) — correct for a multi-tenant SaaS where every business only sees its own rows, but wrong here: this platform has one shared dataset that different roles see different *slices* of, not separate tenants who own separate data.

Five roles (from the Architecture doc's User Roles section):

Role	Can do
General Public	Read published disaster events, risk layers, alerts — no admin-unit restriction, read-only

Role	Can do
Emergency Responder	All public reads + create/update disaster event reports within their assigned region
Government Official	All responder reads/writes within their assigned region + view analytics/dashboards
Researcher	Read access to full historical data (incl. unpublished/under-review), export endpoints
Administrator	Full read/write, user management, data-source management, no region restriction

Region scoping, not tenant isolation: every staff user (`Emergency Responder` , `Government Official`) has a nullable `scope_admin_unit_id` on `users`. `null` = unrestricted (Administrators, and Government Officials with a state-wide mandate); a non-null value restricts that user to actions on data within that `admin_unit` **or any of its descendants**, found by walking `admin_units.parent_id` ancestry:

```
// section 8 has the repository-level recursive CTE this calls
async function assertRegionAccess(user: AuthUser, targetAdminUnitId: string)
{
  if (!user.scopeAdminUnitId) return; // unrestricted
  const isDescendant = await
AdminUnitRepository.Instance.isDescendantOrSelf(targetAdminUnitId,
user.scopeAdminUnitId);
  if (!isDescendant) throw new ForbiddenError();
}
```

List/search endpoints call `applyRegionScope(query, user)` instead, which adds a `WHERE admin_unit_id IN (<scope subtree>)` clause rather than throwing — a scoped user's list results are silently filtered to their region, not rejected.

Authorization guard order: Firebase ID-token verification (via the Firebase Admin SDK) resolves `req.user` → a role guard (`authorise("emergency_responder", "government_official")`) checks the role is permitted for this route at all → the service-level `assertRegionAccess` / `applyRegionScope` checks the specific region. Role check is cheap and happens first; region check needs a DB lookup (ancestry walk) and happens in the service.

Authentication mechanism: Firebase Authentication

Identity and password storage are fully delegated to Firebase Authentication — this backend never hashes, stores, or even sees a password. The client app (web/mobile) uses the Firebase Auth SDK directly for sign-up, sign-in, email verification, and password reset; Firebase handles all of that, including its own hashing and storage, entirely outside this codebase.

What changes in the request flow (section 3): the client attaches a **Firebase ID token** (a short-lived JWT Firebase issues and refreshes automatically) as a **Bearer** token on every API request. The Auth Middleware layer verifies it server-side with the Firebase Admin SDK — not by checking a locally-issued JWT signature:

```
// middleware/authMiddleware.ts
import { getAuth } from "firebase-admin/auth";

export async function authMiddleware(req: Request, res: Response, next:
NextFunction) {
  const header = req.headers.authorization;
  if (!header?.startsWith("Bearer ")) throw new AppError(401, "Missing bearer
token");
  try {
    const decoded = await getAuth().verifyIdToken(header.slice(7));
    req.firebaseUser = decoded; // { uid, email, ... }
    next();
  } catch {
    throw new AppError(401, "Invalid or expired token");
  }
}
```

Postgres stays the source of truth for role and region — not identity. The **users** table keeps **role** and **scope_admin_unit_id** exactly as described above, but replaces any password/hash column with a unique, **NOT NULL** **firebaseUid** column. A resolver step after **authMiddleware** looks up (or, on first sight of a UID, creates) the matching Postgres row and attaches the full **AuthUser** — role, scope, and profile fields — to **req.user**, which is what every example elsewhere in this document (**req.user**, **AuthUser**) actually refers to:

```
// A newly-registered Firebase user has no Postgres row yet on their first
request
// resolveUserMiddleware runs after authMiddleware, before authorise()
async function resolveUserMiddleware(req: Request, res: Response, next:
NextFunction) {
  let user = await
UserRepository.Instance.findByFirebaseUid(req.firebaseUser!.uid);
  if (!user) {
    user = await UserRepository.Instance.create({
      firebaseUid: req.firebaseUser!.uid,
      email: req.firebaseUser!.email!,
      role: "general_public", // lowest-privilege default; an Administrator
elevates it later
      scopeAdminUnitId: null,
    });
  }
  req.user = user;
  next();
}
```

This keeps the access-control model above — five roles, `scope_admin_unit_id` ancestry — completely unchanged; only *how* `req.user` gets populated changes, not what it means once it's there.

8. Repository layer, indexing & geospatial queries

Not in the source guide — new for this project. All database access, including all PostGIS access, goes through a repository. The base contract carries over:

```
export abstract class BaseRepository<T extends ObjectLiteral> {
  protected abstract repo: Repository<T>;
  async findById(id: string): Promise<T | null> { return
this.repo.findOneBy({ id } as any); }
  async findPage(opts: OffsetPage): Promise<{ items: T[]; meta: PageMeta }> {
/* offset pagination */ }
  async findCursorPage(opts: CursorPage): Promise<{ items: T[]; nextCursor?:
string }> { /* keyset pagination */ }
}
```

Geometry columns use TypeORM's `geometry` column type with an explicit SRID:

```
@Column({ type: "geometry", spatialFeatureType: "Point", srid: 4326 })
location: Point;
```

Every geometry column gets a GiST spatial index, created via migration (TypeORM's `@Index` doesn't generate the right index type on its own):

```
CREATE INDEX idx_disaster_events_location ON disaster_events USING GIST
(location);
CREATE INDEX idx_flood_zones_geom ON flood_susceptibility_zones USING GIST
(geom);
```

Common spatial query patterns, written in the repository, never inline in a service:

```
// Which admin_unit contains this point? (point-in-polygon)
async findAdminUnitContaining(lon: number, lat: number, level?: string) {
  const qb = this.repo.createQueryBuilder("au")
    .where("ST_Contains(au.boundary, ST_SetSRID(ST_MakePoint(:lon, :lat),
4326))", { lon, lat });
  if (level) qb.andWhere("au.level_name = :level", { level });
  return qb.getOne();
}

// Disaster events within N km of a point
async findNear(lon: number, lat: number, radiusKm: number) {
  return this.repo.createQueryBuilder("e")
    .where("ST_DWithin(e.location::geography, ST_SetSRID(ST_MakePoint(:lon,
:lat), 4326)::geography, :meters)",
    { lon, lat, meters: radiusKm * 1000 })
    .getMany();
}

// Flood zones intersecting a given boundary
async findZonesIntersecting(adminUnitId: string) {
  return this.repo.createQueryBuilder("z")
    .innerJoin("admin_units", "au", "au.id = :id AND ST_Intersects(z.geom,
au.boundary)", { id: adminUnitId })
    .getMany();
}
```

Rules: every geometry column is `SRID 4326` end to end — no silent reprojection in application code; cast to `::geography` (as above) when a query needs real-world distances in meters rather than `ST_Distance`'s planar degrees;

`ST_Intersects` / `ST_Contains` / `ST_DWithin` are always run in the database, never by loading all rows and checking in JS. See section 10 for the companion rule on what geometry a query is allowed to return.

9. Model / entity layer

This is a greenfield backend, so entities use TypeORM's **decorator style** (the source guide's other codebase used `EntitySchema` objects for historical reasons — not carried over here; decorators are more readable and are TypeORM's documented default for new projects).

Geometry column type is matched to what the Architecture doc's ERD calls for per table — never forced into lat/lon points when the real shape is a line or an area (project brief, section 6):

```
@Entity("admin_units")
export class AdminUnit {
  @PrimaryGeneratedColumn("uuid") id: string;
  @Column() name: string;
  @Column() level: number;
  @Column({ name: "level_name" }) levelName: string; // e.g. "LGA", "State",
  "Community"
  @Column({ name: "parent_id", nullable: true }) parentId: string | null;
  @ManyToOne(() => AdminUnit) @JoinColumn({ name: "parent_id" }) parent:
  AdminUnit | null;
  @Column({ type: "geometry", spatialFeatureType: "MultiPolygon", srid: 4326
  })
  boundary: MultiPolygon;
}

@Entity("disaster_events")
export class DisasterEvent {
  @PrimaryGeneratedColumn("uuid") id: string;
  @Column({ name: "disaster_type_id" }) disasterTypeId: string;
  @Column({ name: "admin_unit_id" }) adminUnitId: string;
  @Column({ name: "event_date", type: "timestampz" }) eventDate: Date;
  @Column({ type: "geometry", spatialFeatureType: "Point", srid: 4326 })
  location: Point; // a disaster event is
  a POINT
  @Column({ name: "data_type" }) dataType: "OBSERVED" | "DERIVED" |
  "MODELLED" | "ESTIMATED" | "SYNTHETIC";
  @Column({ name: "source_id" }) sourceId: string;
```

```

}

@Entity("flood_susceptibility_zones")
export class FloodSusceptibilityZone {
  @PrimaryGeneratedColumn("uuid") id: string;
  @Column({ type: "geometry", spatialFeatureType: "MultiPolygon", srid: 4326 })
  geom: MultiPolygon; // a susceptibility
  zone is an AREA
  @Column({ name: "risk_level" }) riskLevel: "low" | "moderate" | "high" |
  "very_high";
  @Column({ name: "data_type" }) dataType: "OBSERVED" | "DERIVED" |
  "MODELLED" | "ESTIMATED" | "SYNTHETIC";
}

// A river/waterway is a LINESTRING (rivers_waterways table) – same pattern,
spatialFeatureType: "LineString"

```

Every entity's `dataType` / `sourceId` columns are `NOT NULL` at the database level too — the Zod validation in section 4 is the first gate, the DB constraint is the second, so a bug in application code still can't insert an unsourced row.

10. Pagination standards

Offset pagination (`?page=2&pageSize=25`) for admin screens and small, stable lists (admin units, data sources, disaster types). **Cursor/keyset pagination** (`?cursor=<opaque>&limit=25` , ordered by an indexed column such as `event_date`) for large or frequently-changing lists (disaster events, rainfall observations) — offset pagination degrades and can skip/duplicate rows on a table that's being written to continuously.

New rule for this project, not in the source guide: a list endpoint never returns full geometry. Returning a full `MultiPolygon` WKT/GeoJSON on every row of a paginated list is how a 25-row response becomes several megabytes. The rule:

- **List endpoints** (`GET /api/v1/disaster-events` , `GET /api/v1/flood-zones`) return either no geometry field at all, or a **bounding-box summary** only: `{ bbox: [minLon, minLat, maxLon, maxLat], centroid: [lon, lat] }`.
- **Detail endpoints** (`GET /api/v1/disaster-events/:id`) return the full geometry for that one record.

- **Map layer endpoints** (`GET /api/v1/disaster-events/geojson?bbox=...`) are the dedicated route for full-geometry, map-ready data — they still require a `bbox` query filter (rejecting an unbounded request) so a client can't accidentally request every flood zone in the state as full polygons in one call.

This mirrors the flood-zone-as-polygon-not-point distinction from the ERD (project brief, section 6) at the API layer: the shape is preserved where it matters (detail, map layers), and deliberately dropped where it would just be dead weight (lists).

11. API versioning, router mounting & naming conventions

All routes are prefixed `/api/v1/`. REST naming: plural nouns, nesting only one level deep (`/api/v1/admin-units/:id/disaster-events` , not deeper); actions that don't map to CRUD get a verb sub-path (`/api/v1/disaster-events/:id/verify`).

```
GET    /api/v1/admin-units
GET    /api/v1/disaster-events
GET    /api/v1/disaster-events/:id
GET    /api/v1/disaster-events/geojson?bbox=...
POST   /api/v1/disaster-events
GET    /api/v1/rainfall-observations
GET    /api/v1/risk/flood-susceptibility?adminUnitId=...
GET    /api/v1/alerts
GET    /api/v1/data-sources
```

A real production incident, carried over from the source guide because the lesson is domain-agnostic: in the source codebase, every feature router independently called `router.use(authMiddleware, tenantMiddleware)` at its own top, on the assumption each router only runs its own middleware once. But all routers were mounted under one shared prefix in `app.ts` (`app.use("/api/v1", [usersRouter, ordersRouter, ...])`), and Express walks **every** mounted router's stack for **every** matching request — so `authMiddleware / tenantMiddleware` re-ran once per router, not once per request, adding cumulative multi-second overhead as more feature routers were added.

Fix, and the rule going forward: `auth` and any tenant/region-scope-resolution middleware are mounted exactly once, centrally, in `app.ts` , before the routers are attached — never re-declared inside an individual feature router:

```
// app.ts — correct: runs once per request, not once per router
app.use("/api/v1", authMiddleware, resolveUserScopeMiddleware);
app.use("/api/v1/disaster-events", disasterEventsRouter);
app.use("/api/v1/rainfall-observations", rainfallRouter);
// ...
```

A feature router only adds `authorise(<roles>)` for its own route-specific role check — never re-adds `authMiddleware` itself.

12. Feature scaffold checklist

Worked through for a concrete example — adding `disaster_events`:

- Entity** — `disaster-events/disaster-event.entity.ts`: columns, geometry type (`Point`), `dataId` / `sourceId` as `NOT NULL`, relations to `AdminUnit` and `DisasterType`.
- Migration** — create table + `GIST` index on `location` + FK constraints; never edit an already-applied migration, always add a new one.
- Repository** — `disaster-event.repository.ts` extends `BaseRepository<DisasterEvent>`; add `findNear`, `findAdminUnitContaining` callers as needed (section 8).
- Validation schemas** — `disaster-event.validation.ts`: `createDisasterEventSchema`, `listDisasterEventsSchema` (query params: `bbox`, `adminUnitId`, `disasterTypeId`, `dateFrom/dateTo`, `cursor/limit`).
- Service** — `disaster-event.service.ts`: singleton, calls `assertRegionAccess` / `applyRegionScope`, verifies `sourceId` (section 6).
- Controller** — thin, one method per route, all try/catch → `next(err)`.
- Routes** — `disaster-event.routes.ts`: attach validation middleware, `authorise(<roles>)`, wire to controller methods; mounted once in `app.ts` under `/api/v1/disaster-events` (section 11).
- Unit tests** — service layer: region-scope enforcement, `UnprovenDataError` on missing/invalid `sourceId`, geometry validation edge cases.
- Postman collection** — add the module folder with one example response per status code (section 14).
- Typecheck** — `npm run typecheck` clean before opening a PR.
- API docs** — update the endpoint table (section 11) if a new route was added.

12. **Data-provenance review** — confirm the feature has no path that can write `disaster_events` without a validated `dataType` / `sourceId` — the code-level enforcement of the project's core anti-fabrication rule (section 13).
13. **Beginner documentation** — write `docs/<feature-name>.md` following the section 18 template: what was built, how it works, resources to read, and a layman's explanation. A PR is not done until this file exists — this is not an optional last step, it's part of the feature.

13. AI Agent Mandatory Rules

Any AI coding agent (Claude Code or otherwise) working in this codebase follows these rules without exception:

1. **TypeScript strict mode, always.** No `any` without an inline comment explaining why it's unavoidable. Run with `tsx`, never `node`.
2. **`npm run typecheck` before declaring any task done.** A task isn't finished if `tsc --noEmit` reports errors anywhere in the project, not just in touched files.
3. **Never invent data — this is the platform's most important rule, and it applies to code, not just to the data-team's research.** An agent must never:
 - write a seed script, mock, or fallback value into `disaster_events`, `rainfall_observations`, `population_estimates`, or `flood_susceptibility_zones` without a real `sourceId` and correct `dataType`
 - invent plausible-looking coordinates, severities, affected-population counts, or rainfall figures to "fill in" a demo or make a feature look complete
 - relabel `SYNTHETIC` /demo data as `OBSERVED` to make a chart or map look more populated
 - silently modify a value from an external source — corrections are stored as a new `DERIVED` record referencing the original, never an in-place overwrite (project brief, section 7)
4. **`SYNTHETIC` data only via the dedicated demo-data path**, gated behind `NODE_ENV=development` or an explicit `/api/v1/dev/seed` route that does not exist in production builds — never mixed into the same tables/endpoints real data flows through without the tag.
5. **No layer-skipping.** A controller never imports a repository; a service never imports `AppDataSource` directly; all DB access goes through a repository (section 3).

6. **No new middleware re-declared per-router.** Auth/scope-resolution middleware is mounted once, centrally (section 11) — the incident in that section is exactly the bug this rule prevents.
7. **Every geometry write is validated and SRID-tagged 4326** before it reaches the repository (section 4); an agent never accepts or stores geometry in an unknown or unstated CRS.
8. **List endpoints never return full geometry** (section 10) — an agent adding a new list route must apply the bbox/no-geometry rule, not copy a detail endpoint's serializer.
9. **Region-scope checks are never skipped for convenience.** A quick admin script or debug route is not exempt from `assertRegionAccess / applyRegionScope` unless it is explicitly Administrator-only and documented as such.
10. **Uncertain about a data source, a scope decision, or whether something counts as OBSERVED vs DERIVED ?** Ask, or mark it `NEEDS VERIFICATION` and flag it — never guess silently and ship it as if it were settled.
11. **Every implementation ships with its documentation.** No feature, endpoint, or migration is "done" until its `docs/<feature-name>.md` exists per section 18 — a plain-English summary, a reading list for the concepts used, and a layman's explanation of what was built. Code without this documentation is incomplete work, not finished work with docs pending.

14. File uploads (Future) & Postman collection

File uploads — marked Future, deferred out of the MVP. No current backlog item needs user-uploaded files (a disaster-report photo attachment is plausible later, but isn't confirmed MVP scope). The source guide's Cloudinary pattern is documented here so it's ready when needed, not built now:

- `StorageService` is the sole owner of the Cloudinary SDK — no other module imports it directly.
- `multer.memoryStorage()` only — never write an uploaded file to local disk first (irrelevant in most container deployments anyway, and a needless failure mode).
- Three methods: `uploadImage(buffer, folder)`, `deleteImage(publicId)`, `replaceImage(oldPublicId, buffer, folder)`.
- Folder naming rebranded for this project: `climate-platform/disaster-reports`, `climate-platform/evidence-photos`, `climate-platform/user-avatars` (replacing the source guide's `labafood/products` -style naming).

- When this is built: a disaster-report photo becomes evidence attached to a `data_type: OBSERVED` community report, itself reviewed before being treated as a trusted source — not automatically equivalent to a verified `OBSERVED` disaster event.

Postman collection standard — carried over, adapted to this project's modules:

- One folder per feature module (Auth, Admin Units, Disaster Events, Rainfall Observations, Risk Assessments, Alerts, Data Sources), mirroring `src/modules/`.
- Every request includes **one example response per possible status code**, derived mechanically by walking the route's middleware chain and every `AppError` subclass its service can throw — not written from memory (a `disaster-events` create request needs examples for `201`, `400` validation, `401` no token, `403` wrong region, `422` `UnprovenDataError`, `500`).
- **Token chaining** via post-response scripts: the Auth folder's login request writes `pm.environment.set("accessToken", ...)`, and every other folder's requests reference `{{accessToken}}` — no request in the collection has a hardcoded token.
- Environments: `local`, `staging`, `production` (base URL only differs; never store production credentials in the committed collection).

15. Email standards

Fully rebranded from the source guide's orange `#f97316` / dark `#0f172a` LabaFood palette to this project's established brand tokens: **accent teal** `#0E6B6F`, **dark** `#063C3E`, matching the delivered rainfall workbook and boundary-data styling already in use across this project's artifacts.

A single `emailLayout()` wrapper component (logo, footer, unsubscribe/contact links) with a fixed component library used inside it: heading, body text, primary button, info card, divider — no ad-hoc inline styling in individual email templates.

Use cases for this platform (replacing the source guide's food-order/school set):

Verify email and Reset password are handled natively by Firebase Authentication (see section 7's Authentication mechanism), not sent through the relay below — rebuilding what Firebase already does securely would add risk for no benefit. The relay carries the emails specific to this platform's own domain logic, which Firebase has no way to know about:

Email	Trigger	Sent via
Verify email	Account registration	Firebase Authentication (native — not this relay)
Reset password	Password-reset request	Firebase Authentication (native — not this relay)
Region-access request approved	Admin approves a staff user's <code>scope_admin_unit_id</code> request	Apps Script relay · Transactional
Region-access request denied	Admin denies the same	Apps Script relay · Transactional
Data import completed	Scheduled/manual ingestion pipeline run finishes	Apps Script relay · Transactional
Data import failed	Ingestion pipeline run errors out	Apps Script relay · Alerting
[Future] High-severity alert	A <code>severe / catastrophic</code> disaster event is published in a user's scoped region	Apps Script relay · Alerting

Tone guide — narrowed to two tiers. The source guide's three-tier tone system (Transactional / Alerting / Celebratory) is cut to two: **Transactional** (neutral, informative, calm confirmation) and **Alerting** (clear urgency without alarmism — states what happened and what to do, no exclamation-heavy copy). **Celebratory is dropped entirely** — a disaster-management platform has no "congratulations" moments, and forcing warm/celebratory copy into this domain would read as tone-deaf next to alert emails.

Subject-line convention: [Climate Platform] <plain statement>, e.g. [Climate Platform] Verify your email, [Climate Platform] Data import failed — CHIRPS rainfall, 2026-09 — never clickbait phrasing, especially for Alerting-tier email.

Email delivery mechanism: Google Apps Script relay

Email is actually sent via a **Google Apps Script Web App** acting as a thin relay in front of Gmail's `MailApp` service — not a transactional email provider (SendGrid, SES, Postmark) and not raw SMTP. The backend never talks to Gmail directly; it makes one HTTP `POST` to the deployed Apps Script URL, and the script does the actual sending. This script already exists and is deployed — it lives in Google Apps Script, not in this repository, and the agent does not modify it as part of backend feature work.

```
// Deployed separately as a Google Apps Script Web App – reference only, not
// part of this repo
/**
 * Web App: send an email from a JSON POST request.
 * Expected JSON body:
 * { "to": "...", "subject": "...", "htmlBody": "...", "body": "...",
 *   "name": "(optional) sender display name", "replyTo": "(optional) reply
address" }
 */
var DEFAULT_SENDER_NAME = "Test Mate";

function doPost(e) {
  try {
    var data = JSON.parse(e.postData.contents);
    if (!data.to || !data.subject) {
      throw new Error("Both 'to' and 'subject' are required.");
    }
    var options = {
      to: data.to,
      subject: data.subject,
      body: data.body || "",
      name: data.name || DEFAULT_SENDER_NAME
    };
    if (data.htmlBody) options.htmlBody = data.htmlBody;
    if (data.replyTo) options.replyTo = data.replyTo;
    MailApp.sendEmail(options);
    return jsonResponse({ status: "success", to: data.to });
  } catch (err) {
    return jsonResponse({ status: "error", message: err.message });
  }
}

function jsonResponse(obj) {
  return
  ContentService.createTextOutput(JSON.stringify(obj)).setMimeType(ContentService.MimeType.JSON);
}
```

Backend integration — EmailService is the sole owner of this call, following the same pattern as **StorageService** for Cloudinary (section 14): no controller or other service ever calls the relay URL directly.

```
// common/email/email.service.ts
export class EmailService {
  private static _instance: EmailService;
  static get Instance() { return this._instance ??= new
EmailService(process.env.EMAIL_RELAY_URL!); }
  private constructor(private relayUrl: string) {
    if (!relayUrl) throw new Error("EMAIL_RELAY_URL is not set");
  }

  async send(input: { to: string; subject: string; htmlBody: string; name?:
string; replyTo?: string }) {
    const res = await fetch(this.relayUrl, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ ...input, name: input.name ?? "Rivers State
Climate Platform" }),
    });
    const result = await res.json() as { status: "success" | "error"; to?:
string; message?: string };
    if (result.status === "error") throw new
EmailDeliveryError(result.message ?? "Unknown email delivery failure");
    return result;
  }
}

export class EmailDeliveryError extends AppError {
  constructor(message: string) { super(502, `Email delivery failed:
${message}`); }
}
```

`EMAIL_RELAY_URL` (the deployed Apps Script `/exec` URL) is an environment variable — never hardcoded or committed, same rule as the Mapbox token and Cloudinary credentials.

Two limitations to be aware of before this carries anything sensitive:

1. **No authentication on the relay itself.** The Apps Script as written accepts a POST from anyone who has the URL — there's no shared-secret or API-key check in `doPost`. For a platform that will eventually send disaster alerts, an unauthenticated relay is a real risk (anyone with the URL could send email "from" the platform). Before this handles anything beyond low-stakes MVP transactional email (verify email, password reset),

add a shared-secret check in the script itself — reject the request unless the JSON body includes a token matching a value stored in the script's own properties — and have `EmailService` send that token from an environment variable.

2. **Gmail/Apps Script sending quotas.** `MailApp.sendEmail` is subject to Google's daily quota (roughly 100 emails/day on a personal Gmail account, higher on Google Workspace) — fine for the MVP's transactional volume (verify email, password reset, region-access approvals), but not a fit for the `[Future]` high-severity alert email use case once that involves broadcasting to many subscribed users. That's a real constraint on when the alert-email Future item can graduate to Should/Must, not just a feature-flag decision.

`DEFAULT_SENDER_NAME` in the script is a generic placeholder (`"Test Mate"`) — `EmailService.send` always passes an explicit `name` , so the platform's real display name never depends on remembering to edit the deployed script separately.

16. Final principle & what changed from the source guide

SOLID and file-size discipline recap: one responsibility per file; a file pushing past roughly 200–300 lines is a signal to split it by responsibility, not a hard limit to game; no controller, service, or repository file mixes concerns from more than one module.

Summary of every adaptation made to the source guide, for the team's visibility:

Source guide	This project
Flat <code>ownerId</code> multi-tenant isolation	Role (5 roles) + hierarchical region-scoping via <code>scope_admin_unit_id</code> (section 7)
No geospatial section	New section 8: PostGIS columns, GiST indexes, <code>ST_Intersects</code> / <code>ST_DWithin</code> / <code>ST_Contains</code> patterns
No provenance/data-quality validation	New hard rule: <code>dataType</code> + <code>sourceId</code> required on writes to the core data tables (section 4, 9, 13)
<code>EntitySchema</code> entities	Decorator-style TypeORM entities (greenfield choice)
File uploads live and in use	Marked Future/deferred — no confirmed MVP need yet

Source guide	This project
Orange/dark LabaFood email branding, 3 tones	Teal/dark project branding, 2 tones (Celebratory dropped)
Router-mounting incident (§14.1)	Kept as-is — domain-agnostic lesson, re-labelled for this codebase (section 11)

The underlying stack decision (Node.js/TypeScript/Express/TypeORM/PostgreSQL) and the layered-architecture shape, response envelope, and error-handling pattern all carried over largely unchanged — those parts of the source guide were already a good fit.

Section 17 below sequences implementation into 9 phases starting from authentication; the Approach, Architecture & Backlog doc’s own “Backend Implementation Phases” section ties the same phases to specific product-backlog items.

17. Implementation Phases (starting from Authentication)

The build starts from authentication, as requested, and proceeds in 9 phases. Each phase names the sections of this standard that govern it, so an engineer picking up a phase knows exactly which parts of the document to follow. The same 9 phases are tracked against specific product-backlog items in the **Approach, Architecture & Backlog** doc’s “Backend Implementation Phases” section — this table is the implementation-standard view of the identical sequence.

```
flowchart TD
    P1[Phase 1: Foundation and Authentication] --> P2[Phase 2: Admin Geography]
    P2 --> P3[Phase 3: Data Sources and Provenance]
    P3 --> P4[Phase 4: Disaster Taxonomy and Events]
    P3 --> P5[Phase 5: Climate and Rainfall Ingestion]
    P4 --> P6[Phase 6: Risk Assessment]
    P5 --> P6
    P6 --> P7[Phase 7: Alerts]
    P4 --> P8[Phase 8: Community Reports - optional]
    P7 --> P9[Phase 9: Hardening and Launch Readiness]
    P8 --> P9
```

Phases 4 and 5 can run in parallel once Phase 3 lands — they don’t depend on each other, only on `data_sources` existing so their writes have somewhere to point `source_id` at.

Phase	Focus	Key entities/tables	Depends on	Sections of this standard to follow	Definition of done
1. Foundation & Authentication	Project skeleton, Firebase Authentication integration (ID-token verification via the Admin SDK), role model, global error handling, region-scope guard wired in from day one	<code>users</code> , <code>roles</code>	None	§2 Core philosophy, §3 Architecture & folders, §5 Response envelope & errors, §6 Controller/service, §7 Access control	A user can register and log in via Firebase Authentication (client-side), and hit a role-guarded backend route using their Firebase ID token; a first-time token creates the matching Postgres users row; wrong role/region is rejected; <code>npm run typecheck</code> clean; Postman Auth folder using a real Firebase ID token
2. Admin Geography	<code>admin_units</code> hierarchy + boundary geometry seeded from the real, already-acquired LGA data; point-in-polygon lookup. This phase also builds the generic import capability (CSV/GeoJSON/Shapefile/XLSX → staging → validation → promotion; geometry accepted as GeoJSON or WKT — Architecture doc §1), using this LGA import as its first real run.	<code>countries</code> , <code>admin_units</code>	Phase 1	§8 Repository & geospatial queries, §9 Model/entity layer	All 23 LGAs queryable with valid geometry; GiST index in place; <code>ST_Contains</code> point-in-polygon test passes; the generic import path (staging → validation → promotion) is proven end-to-end via this import, using the <code>lga_boundaries</code> sheet in the delivered <code>rivers_state_rainfall_and_boundaries.xlsx</code> workbook — an XLSX file with WKT-format boundary geometry — as the test fixture, parsed without any manual conversion to GeoJSON first.
3. Data Sources & Provenance	<code>data_sources</code> / <code>datasets</code> CRUD, admin-only writes — built before any table that requires a <code>sourceId</code>	<code>data_sources</code> , <code>datasets</code>	Phase 1	§4 Validation layer, §9 Model/entity layer	A source can be created and referenced; a write to a later table without a valid <code>sourceId</code> is rejected
4. Disaster Taxonomy & Events	Controlled taxonomy, disaster event CRUD, region-scoped writes, list/detail/geojson endpoints	<code>disaster_types</code> , <code>disaster_events</code>	Phase 3	§4 Validation, §8 Geospatial queries, §9 Model/entity, §10 Pagination, §12 Scaffold checklist (worked example)	An Emergency Responder can create an event scoped to their region; General Public sees only published events; list responses omit full geometry
5. Climate & Rainfall Ingestion	Reuse the generic import capability built in Phase 2 for a new feed type — the delivered CHIRPS rainfall dataset is the first climate-data feed through it.	<code>rainfall_observations</code> / <code>climate_observations</code>	Phase 3	§4 Validation (dataType/sourceId), §13 AI Agent Mandatory Rules	The delivered CHIRPS CSV imports end to end, tagged <code>OBSERVED</code> , <code>sourceId</code> resolved, zero manual data entry, using the rainfall sheet in that same <code>rivers_state_rainfall_and_boundaries.xlsx</code> workbook as the test fixture

Phase	Focus	Key entities/tables	Depends on	Sections of this standard to follow	Definition of done
6. Risk Assessment	Flood-susceptibility polygons; risk-index computation, formula pending Data Analyst validation	flood_susceptibility_zones , risk_assessments	Phases 4 & 5	§8 Geospatial queries, §9 Model/entity, §13 AI Agent Mandatory Rules	A risk score can be computed and stored per admin unit with full dataType provenance; the formula is documented as provisional
7. Alerts	Alert issuance and status; delivery channel explicitly deferred	alerts	Phase 6	§5 Response envelope & errors, §15 Email standards (Future high-severity alert)	An alert can be created, listed, and tied to a disaster event or risk threshold; delivery integration is a documented Future item
8. Community Reports (optional)	User-submitted disaster reports; only built if confirmed in MVP scope	reports	Phase 4	§14 File uploads (Future, evidence photos)	A report can be submitted and reviewed; never auto-promoted to a verified OBSERVED event without review
9. Hardening & Launch Readiness	Postman collection completed across every module; load-test geospatial queries against the full LGA dataset	—	Phase 7 (& 8 if built)	§11 API versioning & router mounting, §14 Postman collection standard	Full Postman coverage with one example per status code; documented p95 latency for the heaviest geospatial queries

18. Documentation & Beginner Explanations (Mandatory for Every Implementation)

You are writing code that other people — including people newer to this stack than you — will need to read, understand, and maintain. Documentation is not a nice-to-have you add if there's time; it is part of the definition of done for every feature (section 12, step 13) and an AI Agent Mandatory Rule (section 13, rule 11).

For every feature, endpoint, or migration you implement, write a file at `docs/<feature-name>.md` (e.g. `docs/phase-1-authentication.md` , `docs/disaster-events.md`) containing four parts, in this order:

- 1. What was built** — two or three plain sentences: what this feature does, for whom, and why it exists. No jargon here; if you must use a technical term, define it in one clause the first time you use it.
- 2. How it works** — a short walkthrough of the actual files and functions you wrote, in the order a request flows through them (validation → auth → controller → service → repository → database, per section 3). Name the real file paths, not a generic description — the reader should be able to open the files you name and follow along.

3. **Resources to read** — a short reading list of real, official resources for the concepts this feature relies on that the reader may not already know (e.g. JWT authentication, Zod schemas, TypeORM decorators, PostGIS geometry types, GiST indexes). Link to real documentation (official docs, MDN, the library's own README) — never invent a URL or a course that doesn't exist; if you're not certain a resource exists, say so rather than fabricating a link. This is the same anti-fabrication discipline as section 13, applied to documentation instead of data.
4. **Explain it like I'm new to this** — one paragraph explaining the feature the way you'd explain it to someone who can read code in general but hasn't worked with this specific stack or concept before. Use a plain-language analogy where it genuinely helps. Never assume the reader already knows what the stack's specific terms mean.

Worked example — Phase 1 (Authentication), since it's the first phase you'll build.

`docs/phase-1-authentication.md` would look like:

```
# Phase 1: Authentication
```

```
## What was built
```

```
A way for people to create an account, log in, and prove who they are on every
```

```
later request — without this backend ever handling their password. Every request that needs to know "who is this user, and what are they allowed to do" relies on this.
```

```
## How it works
```

```
1. The client app signs up / logs in using the **Firebase Authentication SDK**
```

```
    directly — Firebase hashes, stores, and verifies the password entirely on its own side. This backend never receives a password.
```

```
2. On success, Firebase hands the client a short-lived **ID token** (a signed token, refreshed automatically) that proves who they are.
```

```
3. The client sends that ID token as a `Bearer` token on every API request.
```

```
4. `authMiddleware` (in `middleware/`) verifies the token server-side using the **Firebase Admin SDK** (`verifyIdToken`) — see section 7's Authentication mechanism.
```

```
5. `resolveUserMiddleware` looks up the matching row in our own `users` table by `firebaseUid` (creating one, at the lowest-privilege role, the first time a given user is ever seen) and attaches it to `req.user`.
```

```
6. `authorise(<roles>)` then checks whether `req.user`'s role is allowed on this specific route.
```

```
## Resources to read
```

- Firebase Authentication overview: <https://firebase.google.com/docs/auth>
- Verifying ID tokens server-side:
<https://firebase.google.com/docs/auth/admin/verify-id-tokens>
- Firebase Admin SDK setup (Node.js):
<https://firebase.google.com/docs/admin/setup>
- What a JWT is (Firebase's ID token is one): <https://jwt.io/introduction>

Explain it like I'm new to this

Think of Firebase Authentication as a professional ID-checking service you hire instead of training your own staff to check IDs. When someone signs up or logs in, Firebase's own booth checks their credentials and hands them a wristband (the ID token) – our backend never sees their password, only the wristband. Every door inside our own venue (every API route) just glances at the wristband instead of checking identity from scratch – that's `authMiddleware`, and it's really just asking Firebase's booth "is this wristband real?" The first time a wristband-holder walks through any door, we jot their name down in our own guest book (the `users` table) with the lowest access level, so we know who they are next time; someone with administrator access can later upgrade what they're allowed to do. If someone shows up with no wristband, or a fake one, they're turned away – that's a 401.

Where this lives: a `docs/` folder at the repo root, one file per feature or phase, named to match the phase list in section 17 where possible. This is separate from the Postman collection (section 14, technical/API-shaped) and code comments (which stay technical) — this documentation is deliberately written for a non-expert reader, because the person reading it may be exactly that.